OnCore REST API Overview

2025

Revised January 2025

CONFIDENTIAL

Table of Contents

Summary 3
Architecture 3
Security 3
Implementation and Maintenance 4
Getting Started 4
- Available Resources 4
- API Specification Information 4
- Using Postman 5
  - Create an OAuth client in OnCore 5
  - Perform a POST to get an access token 5
  - Perform a GET using the access token 6
- REST API Client Steps 7
  - Obtain the OnCore REST API Specification 8
  - Download the openapi-generator-cli application 8
  - OpenAPI Generator documentation 8
  - Create a configuration file for code generation 9
  - Run the openapi-generator-cli application to generate the client code in the desired language 9
  - Build the client code as a library 9
  - Add the client library to the interface project 10
  - Add code to authenticate using OAuth access tokens 10
  - OAuth token use 11
  - Additional reading for OAuth access tokens 11

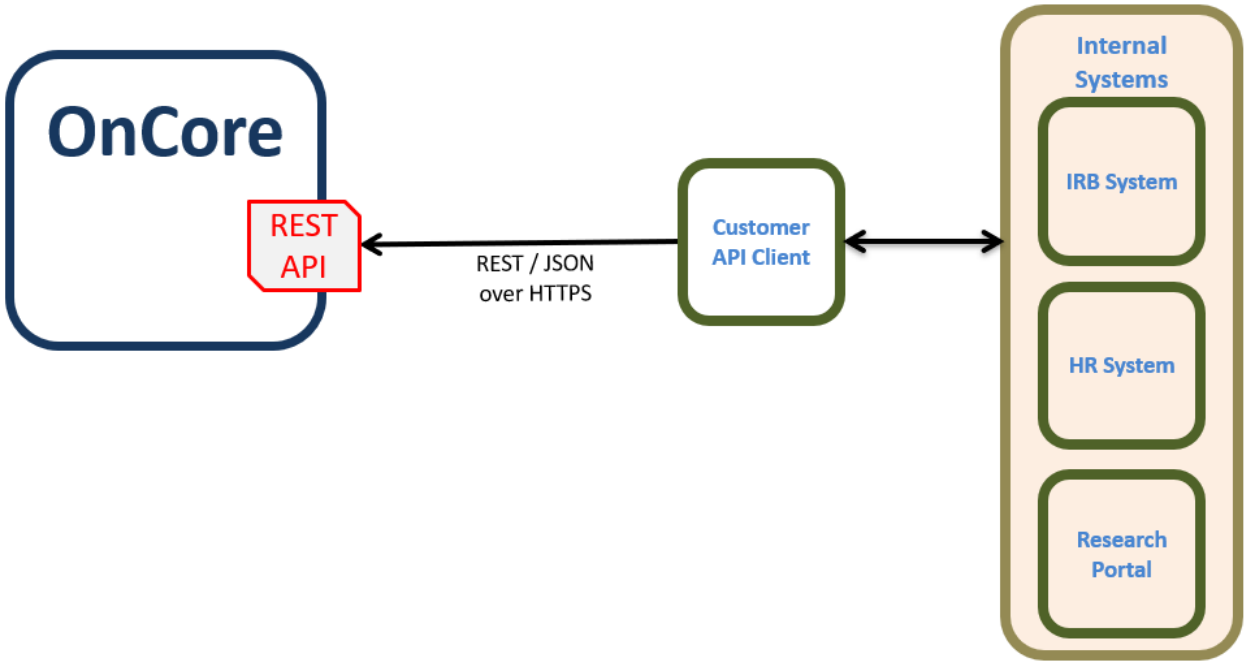
Summary

This document provides an overview of the REST-based (Representational State Transfer-based) OnCore API and its technical infrastructure, as developed and provided by Advarra. Readers of this document are assumed knowledgeable in the areas of web-based technologies.

The API exists to facilitate the exchange of data between OnCore and other applications or systems.

Architecture

The REST-based API employs a design based on some of the key principles from the representational state transfer architectural style and utilizes Hypertext Transfer Protocol Secure (HTTPS) as the transport protocol. The API is deployed within OnCore in the same application server instance.



Security

Programmatic access to any resources within the API requires authentication.

Authentication for the REST-based API must be done using Open Authorization (OAuth). The API provides client authorization via the OnCore application's user roles and permissions functionality. An OAuth client must be built in OnCore with a configured user role. API operations require certain permissions to be assigned to the role of the OAuth client. Refer to the REST API Specification for details on the required permissions for each operation. In addition, the OAuth client must have the correct organization access to be able to perform the operations required by the API client. Refer to the Learning Portal for more information on how to create OAuth clients in OnCore.

All API service operation exchanges are encrypted using Secure Socket Layer (SSL), the same technology used to encrypt all web browser-based communication with the OnCore application.

Implementation and Maintenance

Customers are responsible for developing, testing, and maintaining the API client. New versions of OnCore may require code changes to the API client. Time and resources need to be allocated when performing OnCore upgrades to ensure workflows designed around the API continue to work as expected.

Getting Started

The API uses the OpenAPI specification to define the data model and interface endpoints that are available in the REST-based API. The use of OpenAPI allows for the use of various code generation tools including OpenAPI Generator and Swagger Codegen. Both are open-source projects that allow generation of an API client from an OpenAPI specification.

https://github.com/OpenAPITools/openapi-generator/blob/master/README.md

https://swagger.io/tools/swagger-codegen/

These code generators can use the REST-based API specification to generate client code in a variety of languages (Java, C#, Python, Ruby, etc.).

Available Resources

On the Learning Portal's API Documentation page, there are sample applications available for Java and C#. These can help jump-start development by providing references for how to use the various resources in the REST-based API. The sample applications were first developed using Swagger Codegen, but with the release of OnCore 2024R3, the latest versions of the applications now use OpenAPI Generator. These sample applications also include information on how to set up OAuth for the client application.

The Learning Portal includes a link to a .pdf of the REST API Specification. Below are some notes regarding how to use the REST API Specification and general information applicable across the various resources.

API Specification Information

The records returned from a search are not ordered.

Date fields should be in ISO 8601 format.

Search parameters for GET operations and reference list lookups are case insensitive unless noted otherwise in the specification.

A PUT will update all fields that are not read-only. If a field that is not read-only is not included in a PUT, it will be handled as a null value in the update. If a read-only field is included in a PUT, it will be ignored. Recommended practice for doing updates is to first perform a GET, then update the specific fields that should be updated, and finally do a PUT with that data.

A PUT for an object that contains an array will remove all existing items in the array when the array included in the request is empty. (This is a complete overwrite.)

Using Postman

Postman is a software application that can be used to exercise the OnCore REST API. Postman is available here: https://www.postman.com/

The steps below show how to use Postman to do a basic GET of a protocol from OnCore. The instructions were put together for Postman v8.6.1.

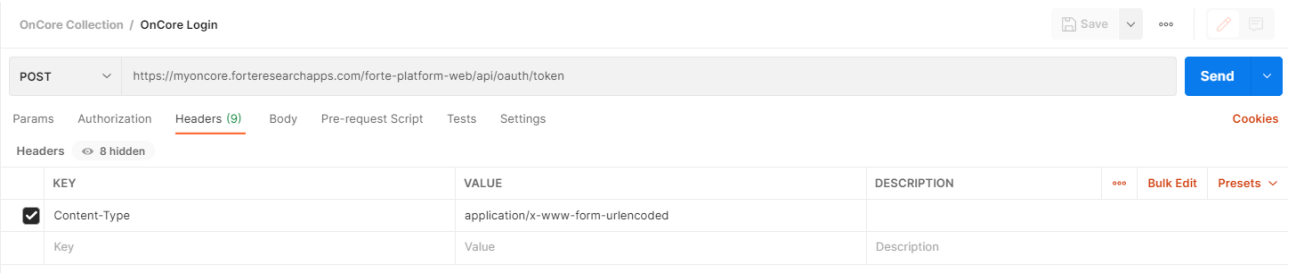
Create an OAuth client in OnCore

Refer to the Learning Portal for information on how to create OAuth clients in OnCore.

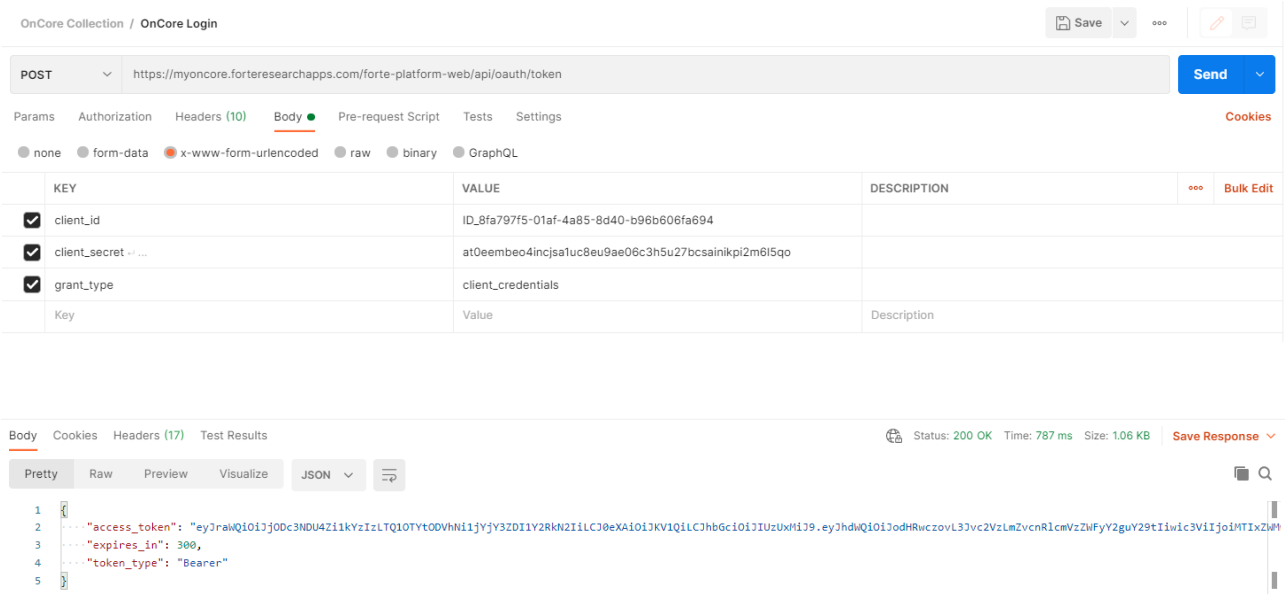
Make note of the client identifier and client secret. These will be used in Postman.

Perform a POST to get an access token

1. In Postman, create a collection to put the requests in (File > New > Collection). A collection is simply a folder in Postman to collect similar requests.
2. From within the new collection, select “Add a request”.
3. Name the request “OnCore Login”.
4. Change the action to POST.
5. Enter the URL for which to obtain an access token. The URL will be a combination of the main OnCore URL that is used to login: for example, **https://myoncore.forteresearchapps.com** followed by **/forte-platform-web/api/oauth/token**
   In this case, the URL would be:
   https://myoncore.forteresearchapps.com/forte-platform-web/api/oauth/token
   Enter the URL in the Request URL section.
6. Add a header with Key Content-Type and Value application/x-www-form-urlencoded
7. Change the Body to x-www-form-urlencoded
8. Add a Key named client_id to the Body with Value equal to the client identifier for your OAuth client.
9. Add a Key named client_secret to the Body with Value equal to the client secret for your OAuth client.
10. Add a Key named grant_type to the Body with Value client_credentials
11. Click Send to obtain an access token.
12. The Body of the response will contain the access token. The access token is valid for a time period depending on the value for Token Lifetime for the OAuth client.



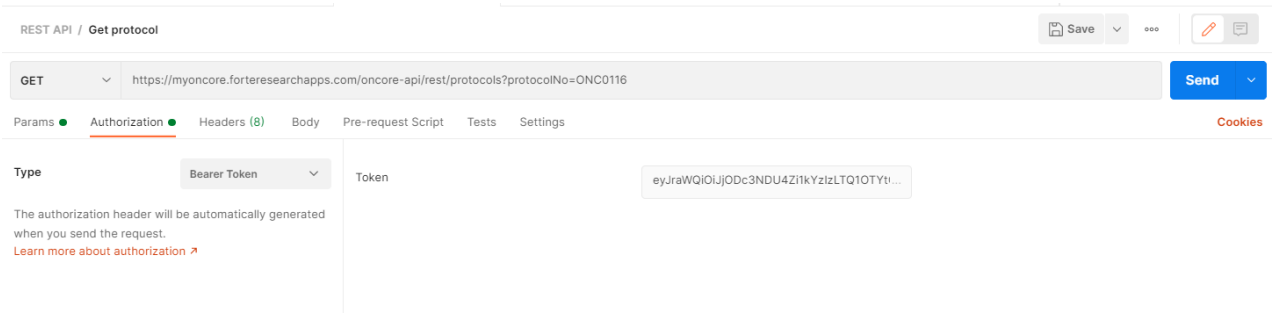
CONFIDENTIAL



Perform a GET using the access token

For this example, we will do a search for an existing protocol.

1. From the collection, right-click and select “Add request”.
2. Name the request “GET Protocol”.
3. Leave the action as GET.
4. Enter the URL for which to perform a GET on a protocol. The URL will be a combination of the main OnCore URL that is used to login: for example, **https://myoncore.forteresearchapps.com** followed by the BasePath as defined in the OnCore REST API Specification: **/oncore-api/rest** followed by the URL for searching for a protocol, as defined in the OnCore REST API Specification: **/protocols**
   In this case, the URL would be:
   https://myoncore.forteresearchapps.com/oncore-api/rest/protocols
   Enter the URL in the Request URL section.
5. On the Authorization tab, change the Type to Bearer Token.
6. For the Token value, enter the access_token obtained above.
7. On the Params tab, add a Query Param with Key protocolNo and Value equal to a known Protocol No in the OnCore environment.
8. Click Send.
9. The Body of the response will contain the protocol information.



CONFIDENTIAL

REST API / Get protocol

Save

GET https://myoncore.forteresearchapps.com/oncore-api/rest/protocols?protocolNo=ONC0116 Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION	Bulk Edit
protocolNo	ONC0116		
Key	Value	Description	

Body Cookies Headers (17) Test Results Status: 200 OK Time: 164 ms Size: 3.09 KB Save Response

Pretty Raw Preview Visualize JSON


```
[
    {
        "protocolId": 27261,
        "protocolStatus": "OPEN TO ACCRUAL",
        "protocolNo": "ONC0116",
        "library": "Oncology",
        "department": "Oncology",
        "organizationalUnit": "Cancer Center",
        "protocolType": "Treatment",
        "nctNo": "NCT00324805",
        "title": "A Phase III Randomized Trial of Adjuvant Chemotherapy With or Without Bevacizumab for Patients With Completely Resected Stage IB (&#8805; 4 cm) - IIIA Non-small Cell Lung
        "shortTitle": "Chemotherapy With or Without Bevacizumab in Treating Patients With Stage IB, Stage II, or Stage ZZZ",
        "objectives": "This randomized phase III trial studies chemotherapy and bevacizumab to see how well they work compared to chemotherapy alone in treating patients with stage IB, stag
        "phase": "III",
        "scope": "National",
        "age": "Adults",
        "investigatorInitiated": true,
        "investigationalDrug": "No",
        "investigationalDevice": "No",
        "drugAccountability": "No",
        "dataMonitoring": "External",
        "includesSpecimenBanking": false,
        "dataTable4ReportType": "Interventional",
        "multiSiteTrial": true,
        "excludeProtocolOnWeb": false,
        "summaryAccrualInfoOnly": false,
        "involvesTherapy": "Yes",
        "precisionTrial": false,
        "precisionTrialClassification": null,
        "pilot": null.
```


REST API Client Steps

The following sections assist in generating client code that can interact with the API. These steps now utilize openapi-generator. In previous versions of this document, steps for using swagger-codegen were described.

The listed steps are required to generate client code for the REST-based API and use it in a project. They are described in greater detail in the following sections.

1. Obtain the OnCore REST API Specification (a.k.a. openApi.yaml). Note that starting with the OnCore 2024R3 release, the specification file extension has been changed to .yaml. It previously had a .yml extension.
2. Download the openapi-generator-cli application.
3. Create a configuration options file for code generation.
4. Run the openapi-generator-cli application to generate the client code in the desired language.
5. Build the client code as a library.
6. Add the client library to the interface project.
7. Add code to authenticate using Open Authorization (OAuth) access tokens.
8. Write code specific to the interface project being worked on.

Obtain the OnCore REST API Specification

The OnCore REST API Specification (also known as openApi.yaml) can be obtained from the OnCore server by downloading: https://yourinstitution.youroncoreinstance.tld/oncore-api/api-spec/openApi.yaml

Note that tld stands for top-level domain and is likely either .app or .com, depending on your specific OnCore instance.

If you are not already logged in to OnCore, you are directed to the login screen and need to enter valid credentials to download the specification.

Download the openapi-generator-cli application

To generate template code from the openApi.yaml file downloaded from the OnCore server, you need to download a copy of the Java-based openapi-generator-cli application of an appropriate version.

The list of releases of openapi-generator-cli is available here:
https://mvnrepository.com/artifact/org.openapitools/openapi-generator-cli

Download openapi-generator-cli.

The rest of this guide refers to the openapi-generator-cli JAR without the version-number. Rename the file to match or edit the commands as needed.

OpenAPI Generator documentation

Online documentation for the OpenAPI Generator is available here: https://github.com/OpenAPITools/openapi-generator/blob/master/README.md

The openapi-generator-cli application can generate client code from the OnCore REST API OpenAPI specification for many different languages.

The list of supported languages for API clients is available here: https://github.com/OpenAPITools/openapi-generator/blob/master/docs/generators.md, as well as specific help for each language.

The rest of this guide is written with the assumption that Java is the client language of choice.

Create a configuration file for code generation

Create a file named config.yaml that resembles the following. Change package names and other details as desired.

Example config.yaml

```
generatorName: java
outputDir: myApiClient
library: native
inputSpec: openApi.yaml
```

Run the openapi-generator-cli application to generate the client code in the desired language

After the code generation config file has been created, the openapi-generator-cli application can be used to convert the OpenAPI specification into a project template by running:

```
java -jar openapi-generator-cli.jar generate --config config.yaml
```

The openapi-generator-cli application will generate a new project directory with both Maven and Gradle build files. Documentation for the data model and the API endpoints will be generated in the ‘docs’ directory. Source code will be placed in ‘src’. A README.md file will also be generated with some basic information on how to build, install, and use the generated API code.

Build the client code as a library

The generated Java client library can be built using the Maven build system by running the following:

```
cd myApiClient
mvn clean install
```

The above will build the client library in the ‘target’ folder and install it to the local machine's Maven repository so that it can be used as part of a larger application/project.

If using Gradle, the following can be used to build the library (in ‘oncore-api-client/build/libs/’):

```
cd myApiCloient
./gradlew jar
```

Add the client library to the interface project

The README.md file within the generated project includes instructions on how to use the generated API client in a program.

In the case of Java, building the API client project will create a JAR library that can be used as part of a Java program that interacts with the OnCore REST-based API. Code will need to be written to perform the API operations that are desired for the specific interface project being developed.

For example, a simple client class that searched for and printed protocols with a given protocol name could be as follows:

```
package org.openapitools.client;

import org.openapitools.client.api.ProtocolsApi;
import org.openapitools.client.model.Protocol;
import java.util.List;

public class OnCoreSampleClient {
    public static void main (String[] args) {
        ApiClient defaultClient = Configuration.getDefaultApiClient();
        defaultClient.setBasePath("https://youroncoreserver.yourinstitution.tld/oncore-api/rest ");

        // You'll need to add code to retrieve a valid time-limited OAuth token and
        // bind it to the client. See below sections for more information.
        defaultClient.setRequestInterceptor(
                r -> r.header("Authorization", "Bearer " + "VALID_OAUTH_TOKEN_VALUE_GOES_HERE"));

        ProtocolsApi apiInstance = new ProtocolsApi();
        String protocolNo = "PROTOCOL NUMBER TO SEARCH FOR";

        try {
            List<Protocol> result = apiInstance.getProtocols(protocolNo, null);
            System.out.println(result);
        } catch (ApiException e) {
            System.err.println("Exception when calling ProtocolApi#getProtocols");
            e.printStackTrace();
        }
    }
}
```

Add code to authenticate using OAuth access tokens

For the API client to communicate with the API, code also needs to be included that obtains a valid OAuth access token and binds it for each request.

The example projects on the Learning Portal contain sample Java and C# code that can be used to programmatically acquire and use OAuth access tokens.

An OAuth token can be created by POSTing an access token request to the OnCore URL:

https://youroncoreserver.yourinstitution.tld/forte-platform-web/api/oauth/token

with `Accept` and `Content-Type` headers of `application/json` and contents of:

```
{
    "client_id" : "CLIENT_ID_FOR_YOUR_OAUTH_CLIENT",
    "client_secret" : "CLIENT_SECRET_FOR_YOUR_OAUTH_CLIENT",
    "grant_type" : "client_credentials"
}
```

The response from an access token creation request will look like the following:

```
{
    "access_token": "YOUR_ACCESS_TOKEN_VALUE",
    "expires_in": 300, "token_type": "Bearer"
}
```

In the above response, the `access_token` is the actual token value issued. `expires_in` is the number of seconds that this access token is valid for from the time it was created. The `token_type` indicates that this can be used as part of an `Authorization` request header with a type of "Bearer".

OAuth token use

Once an OAuth access token has been successfully created, it can then be used to access any OnCore REST API endpoints for which the OAuth client has been granted access.

Requests to OnCore REST API resources must include the following header (with an appropriate value substituted for `YOUR_ACCESS_TOKEN_VALUE`):

```
Authorization: Bearer YOUR_ACCESS_TOKEN_VALUE
```

The API will validate that the token provided is valid and has not expired and will log the OAuth client into OnCore for the duration of the request. If multiple requests are going to be made in succession, each of the requests must have an `Authorization` header present with a valid OAuth access token present.

Additional reading for OAuth access tokens

Please refer to the following locations to learn more about OAuth:

https://www.oauth.com/oauth2-servers/access-tokens/client-credentials/

https://www.oauth.com/oauth2-servers/access-tokens/access-token-response/

While there are multiple workflows in which an OAuth access token can be generated, the method OnCore supports is client_credentials grant.